

**CISS450: Artificial Intelligence
Assignment 10**

OBJECTIVES

1. Solve CSP problems.
2. To be acquainted with standard AI problem(s).

The goal is to develop a CSP solver and use it to solve sudoku and the n -queens problem.

Watch these demo:

- <https://photos.app.goo.gl/2261NhSHHSZ3Ezpa6>
- <https://photos.app.goo.gl/tKyYA7Pevk6SGTcWA>

There are many techniques in solving CSP. Note that my backtrack CSP algorithm is slightly different from the standard one. So follow what was mentioned in class carefully and in my notes.

Q1. You are given a collection of files.

First thing to do is to write a regular brute force backtracking search. (This has no CSP optimizations or heuristics. It's just backtracking.) This was already done in class. Here's a test run. Run make:

```

0: 8.....36.....7..9.2...5...7.....457.....1...3...1...68.85...1..9....4..
1: 98.7.....7.....6....6.5.....4...5.3...79..5.....2...1..85..9.....1...4....3.2.
2: 98.7.....6.....87...7.....5.4...3.5...65...9.....2...1..86...5.....1.3.....4..2
3: 12..3...4...1.2...52..1..5..4..2.....6..7.....3..8.5...9...9.7..3.....8..6
4: .....39.....1..5..3.5.8...8.9...6.7..2...1..4.....9.8..5..2...6..4..7.....
5: .2.4...8.....8...68....71..2..5...9..95.....4..3.....1..7..28...4....6.3..
6: .....1....23.45..51..2...25...1..6...27..8...9.....42...7.3...6...9...8...
7: 12.3.....4.5...6...7.....2.6...1..3...453.....8..9...45.1.....8.....2..7
8: 5.6...7...1.3.....8...5.9....1..2.....8.6.7.....2.4.7...9...6.3...42...5.....
9: ..3..6.8...1..2.....7...4..9..8.6..3..4...1.7.2....3....5.....5...6..98....5.
10: .....12.....345..3..46...2..1.3..7..6...8..9.....5..2..4.6..8...9..7.....
11: 1.....9..67...2..8...4.....75.3...5..2...6.3.....9...8..6...4...1..25...6.
12: 1...6.8...7..1.....5.6..9.4....7.2...3.8...76..3...1..5.4.9.....2.7....
13: 98.76...54.....7..59..4...75....3..2....1...6.9...87...4...1...2...3
14: .2...67..4...8.....93.....9..57..1...7..2.....61.3...4..6...8.....6...5.2.
15: 98.7.....6...5.....4..93..5.....6..7..8.....9..24...1....4...9.1.....1.32
16: 2.....6.5..8..1...4...9...7.3.1.....82.....7.5.3...9...4...8..1..5.6.....2
17: 1.....2..94...5..6...7....89..4...3.6....8.4....2...1..7.....6..5.8..3.
18: 6.....2.9.4...5...1...7...5..84.....2.....3.5.4.2....6...3...9.8...7....1
19: 1.....2.9.4...5...6...7...5.3.4.....6.....58.4...2...6...3...9.8.7.....1
20: ..4...89..7..92.....3...526...1....19....7....1..5...9..4...6..29.....8...3
21: .....94....92.....7..45..1.3....7.6..9..8...7..2.3.7..8...6.1...9...5.2.
22: .....1....2.3...4.5.6...7.8....1.....96..4..7...85.....59...3.2.7...6.5..
index of above sudoku strings or your own sudoku string: 0
view (t-text, g-gui): t
[0] bt0 [2] bt2: 0

```

Here's a quick explanation of the inputs.

- I have included 23 sudoku problems. You can find lots of sudoku problems on the web. Index 1-22 are known to be difficult sudoku puzzles. You can enter an index (0-22) to choose one of them or you can enter a sudoku string (similar in format to given sudoku puzzles). In the above, I choose the sudoku problem at index 0.
- For view, I choose t for console output. You can also choose g for GUI.
- I choose 0 to run bt0 function (the brute force backtrack search that is provided as an example.)

The program then outputs the following:

```

+-----+-----+-----+
| 8 0 0 | 0 0 0 | 0 0 0 |
| 0 0 0 | 0 0 0 | 0 0 0 |
| 0 0 0 | 0 0 0 | 0 0 0 |
+-----+-----+-----+
| 0 0 0 | 0 0 0 | 0 0 0 |
| 0 0 0 | 0 0 0 | 0 0 0 |
| 0 0 0 | 0 0 0 | 0 0 0 |
+-----+-----+-----+
| 0 0 0 | 0 0 0 | 0 0 0 |
| 0 0 0 | 0 0 0 | 0 0 0 |
| 0 0 0 | 0 0 0 | 0 0 0 |
+-----+-----+-----+
states examined: 1
current number of assigned variables: 1 (of 81)
wall clock time: 0.1964 seconds
user time: 0.2090 seconds
system time: 0.0000 seconds

+-----+-----+-----+
| 8 1 0 | 0 0 0 | 0 0 0 |
| 0 0 0 | 0 0 0 | 0 0 0 |
| 0 0 0 | 0 0 0 | 0 0 0 |
+-----+-----+-----+
| 0 0 0 | 0 0 0 | 0 0 0 |
| 0 0 0 | 0 0 0 | 0 0 0 |
| 0 0 0 | 0 0 0 | 0 0 0 |
+-----+-----+-----+
| 0 0 0 | 0 0 0 | 0 0 0 |
| 0 0 0 | 0 0 0 | 0 0 0 |
| 0 0 0 | 0 0 0 | 0 0 0 |
+-----+-----+-----+
states examined: 2
current number of assigned variables: 2 (of 81)
wall clock time: 0.3704 seconds
user time: 0.3350 seconds
system time: 0.0000 seconds
etc.

```

If you choose to run `bt2` (which you have to complete in Q2), the program prompts you for options of whether to you want to apply various optimizations and heuristics such as FC, NC, AC, etc.:

```

0: 8.....36.....7..9.2...5...7.....457.....1...3...1...68.85...1..9....4..
1: 98.7.....7.....6.....6.5.....4...5.3...79..5.....2...1..85..9.....1...4.....3.2.
2: 98.7.....6.....87...7.....5.4...3.5...65...9.....2...1..86...5.....1.3.....4..2
3: 12..3...4...1.2...52..1..5..4..2.....6..7.....3..8.5...9...9.7..3.....8..6
4: .....39.....1..5..3.5.8...8.9...6.7...2...1..4.....9.8..5..2...6..4..7.....
5: .2.4...8.....8...68...71..2..5...9..95.....4..3.....1..7..28...4.....6.3..
6: .....1...23.45..51..2...25...1..6...27..8...9.....42...7.3...6...9...8...
7: 12.3....4.5...6...7.....2.6..1..3...453.....8..9...45.1.....8.....2..7
8: 5.6...7...1.3.....8...5.9...1..2.....8.6.7...2.4.7...9...6.3...42...5.....
9: ..3..6.8...1..2.....7...4..9..8.6..3..4...1.7.2....3....5.....5...6..98....5.
10: .....12.....345..3..46...2..1.3..7..6...8..9.....5..2..4.6..8...9..7.....
11: 1.....9..67...2..8...4.....75.3...5..2...6.3.....9...8..6...4...1..25...6.
12: 1...6.8...7...1.....5.6..9.4....7.2...3.8...76..3...1..5.4.9.....2.7....
13: 98.76...54.....7..59..4...75.....3..2.....1..6.9..87...4...1...2...3
14: .2...67..4...8.....93.....9..57..1...7..2.....61.3...4..6...8.....6...5.2.
15: 98.7.....6...5.....4..93..5.....6..7..8.....9..24...1....4...9.1.....1.32
16: 2.....6.5..8..1...4...9...7.3.1.....82.....7.5.3...9...4...8..1..5.6.....2
17: 1.....2..94...5..6...7.....89..4...3.6.....8.4...2...1..7.....6..5.8..3.
18: 6.....2.9.4...5...1...7...5..84.....2.....3.5.4.2....6...3...9.8...7....1
19: 1.....2.9.4...5...6...7...5.3.4.....6.....58.4...2...6...3...9.8.7.....1
20: ...4...89..7..92.....3...526...1....19....7....1..5...9..4...6..29.....8...3
21: .....94....92.....7..45..1.3....7.6..9..8...7..2.3.7..8...6.1...9...5.2.
22: .....1.....2.3...4.5.6...7.8....1.....96..4..7...85.....59...3.2.7...6.5..
index of above sudoku strings or your own sudoku string: 0
view (t-text, g-gui): t
[0] bt0 [2] bt2: 2
FC? (y/n): y
NC check? (y/n): y
AC check? (y/n): n
variable selection heuristic ([M]RV-MD [n]one): M
value selection heuristic ([L]CV [n]one): n

```

In the above, I have chosen to include FC, NC (but no AC) and I choose the MRV-MD heuristic (but not the LCV heuristic). Note that at this point, since you have not completed the function `bt2` yet, `main.py` will halt with an error. For now don't worry about `bt2`.

`bt0.py` contains the `bt0` brute force backtrack search. This is provided as an example of backtrack. `bt0` is complete. Although it's slow (because it's brute force), you should study it carefully since the final CSP solver is basically a backtracking search algorithm with various optimizations and heuristics and I have included comments that will be helpful in seeing this backtracking search in the context of the CSP solver. The file `bt2.py` has a skeleton of the CSP solver (which is essentially a backtracking search) – but it's incomplete.

Both the `bt0` and `bt2` also accept two extra parameters

- **view**: This object is for visualization. The visualizations are provided, one for console text output and one for GUI. (Refer to `SudokuView`, `SudokuTextView`, `SudokuGUIView` classes in the appropriate files – You do not need to read these file.) The classes are already written and the call `view.run()` is already written in `bt0` and `bt2`, so you don't really need to worry about `view`. However for debugging, you might want to rely on the text view for debug tracing.
- **option**: This is to hold options during the execution of the backtrack search. The CSP backtrack search includes optimizations such as forward checking (FC), node consistency checks (NC), arc consistency checks (AC) and various heuristics (MRV and Max Degree for selecting variables and LCV for selecting values from domain to be assigned to a chosen variable.) For experimentation, when `main.py` is executed, the program asks if the user wants to include FC, NC, etc. Therefore the user can choose to include FC but not NC or AC. Refer to `main.py` for details. Study it carefully.
 - `option['FC']` is boolean
 - `option['NC']` is boolean
 - `option['AC']` is boolean
 - `option['var_selection']` is 'MRV-MD' (i.e., MRV and Max Degree heuristic) or `None`
 - `option['val_selection']` is 'LCV' (i.e. LCV heuristic) or `None`

The other files are:

- **CSP.py**: This contains the class `CSP`. This is more or less the same CSP class that I wrote in class. This is the main class for all CSP problems. Objects of `CSP` will have variables, domains, and constraints. These are defined in the `CSP` class. You will be adding methods to this class. For instance for the CSP solver, you need to select a variable for assignment – the method should probably be in this class. (You can write it as a function outside the class if you prefer.) Similarly you will need to return the domain of a variable ordered according to the LCV heuristic. This can be a method in the `CSP` class as well.
- **SudokuCSP.py**: This contains `SudokuCSP` which is a subclass of `CSP`. Changes to `SudokuCSP` should be minimal since we want our CSP solver to be a general CSP solver (and not just for sudoku puzzles.) The only reason that we have a `SudokuCSP` class is to create the variables, domains, constraints that will be send to the `CSP` class.

Here are the other files:

- **makefile**: Execute `make` at your linux shell to run `main.py`. You do not need to read this file.
- **main.py** will ask the user for inputs and then call `bt0` or `bt2`.
- **csp_util.py**: Some very basic utility functions. These were already discussed in class.
- **sudoku_puzzles.py**: Contains some sudoku puzzles. `main.py` displays about 20 of the puzzles in this file and allows the user to select one of them for testing. You do not need to read this file.

You are advised to study my CSP notes carefully. Most of the work is in writing/modifying `CSP.py` and `bt2.py`. You will need to write another backtrack search function `bt2` in `bt2.py` that contains the algorithm and heuristics for CSP backtrack search.

SUDOKUCSP CLASS

Here are more details about the SudokuCSP class.

The SudokuCSP class allows you to create CSP objects specifically for sudoku problem:

```
class SudokuCSP(CSP):  
  
    def __init__(self, vars=None, dom=None, constraint=None, **arg):  
        ...
```

so that the variables are A1, A2, ..., A9, B1, B2, ..., I1, I2, ..., I9. (A, B, C, ... are the rows and 1, 2, 3, ... are the columns). All variables have domain [1, 2, 3, ..., 9]. The cells which are filled are entered as parameters into the constructor. You can also construct a SudokuCSP problem by passing in vars, dom, constraint just like when you create a CSP object.

For instance

```
sudoku_csp1 = SudokuCSP(A1=7, A6=3, A7=5, A8=1,  
                        B1=5, B2=3, B4=1, B5=8, B6=6,  
                        C3=1, C5=2, C6=7,  
                        D1=8, D5=3, D7=1, D8=4, D9=2,  
                        E1=4, E9=3,  
                        F1=3, F2=6, F3=9, F5=4, F9=8,  
                        G4=9, G5=6, G7=8,  
                        H4=3, H5=1, H6=2, H8=6, H9=5,  
                        I2=4, I3=6, I4=8, I9=9)
```

means that A1 is fixed with value 7, etc.

Recall that ****arg** parameter in a function/method allows you to enter key-value pairs into a function. (See Python notes.) Run this as an example:

```
def f(x, y, **arg):  
    print x  
    print y  
    print arg # this is a dictionary  
  
f(x=0, y=1, A1=7, A6=3)
```

ASSIGNMENTS

Of course you need the concept of assignment. This was already discussed in class.

For simplicity, we'll use a list of tuples. For instance `[('B', 'g'), ('C', 'r'), ('D', 42)]` models the assignment

```
B = 'g'  
C = 'r'  
D = 42
```

You will need to add `(X, x)` to your assignment python list at the tail and you will need to remove it as well. To add you execute the `append` method. To remove you execute the `pop` method.

BOOLEAN EXPRESSION STRINGS AS CONSTRAINTS AND THEIR EVALUATION

We also need boolean expressions for constraints. This was already discussed in class.

For simplicity, we will use Python boolean expression strings. Fortunately, Python has a function to evaluate any valid Python expression.

If you have a Python boolean expression string such as `"B != 'r' and C != 'b'"`, you want this to evaluate to `True` with the substitution `[('B', 'g'), ('C', 'r')]`, because after the assignments

```
B = 'g'
C = 'r'
```

the boolean expression `"B != 'r' and C != 'b'"` evaluates to `False`.

You will need to know the following two functions in python. Go ahead and run the following:

```
x = eval("1 + 2")
print(x)
y = eval("x + 5")
print(y)
exec("z = x + y")
print(z)
```

Make sure you understand the above before proceeding.

Refer to `csp_util.py` for some useful functions:

```
def evaluate(bool_expr_str, assignment):
    # Returns True/False if bool_expr_str evaluates to True/False
    # using assignment
    ...
    return [some boolean value]
```

There are cases where the expression cannot be evaluated to a boolean value. For instance the boolean expression string `"B != 'r' and C != 'b'"` is undefined with the assignment `[('B', 'b')]` since `C` is not defined.

```
def evaluate2(bool_expr_str, assignment):
    # Returns True/False if bool_expr_str evaluates to True/False
    # using assignment that assigns to all variables appearing
    # in the bool_expr_str. Otherwise True is returned.
    ...
    return [some boolean value]
```

Make sure you test and experiment with this simple function thoroughly.

Here are some test cases to get your started:

```

print(evaluate("x == 1", [('x', 1)]))
print(evaluate("x == 1", [('x', 1), ('y', 2)]))
print(evaluate("x == 1", [('x', 2)]))

try:
    print(evaluate("x == 1", [('y', 1)]))
except NameError as e:
    pass

print(evaluate("x == '1'", [('x', '1')]))
print(evaluate("x == '1'", [('x', '2')]))

```

You should read the `csp_util.py` file carefully.

ASIDE. The above hints at programming with logic. Because our CSP is simple, we use Python's ability to evaluate python expressions and statements instead of using a full blown logic library. But beyond simple CSP, once the problem is more complicated, you would need to use a logic library. For instance here's the usage of a logic library

```

>>> from PropositionalLogic import *
>>> A1 = Variable("A1")
>>> A2 = Variable("A2")
>>> P = NE(A1, A2)
>>> print(P)
(A1 != A2)
>>> vars = variables(P)
>>> for var in vars: print var
...
A1
A2
>>> ONE = Constant("1")
>>> assignment = Env([(A1, ONE)])
>>> print(assignment)
[A1->1]
>>> Q = P.substitute(assignment)
>>> print(Q)
(1 != A2)
>>> assignment = Env([(A2, ONE)])
>>> R = Q.substitute(assignment)
>>> print(R)
(1 != 1)
>>> S = R.simplify()
>>> print(S)
FALSE

```

This propositional logic library is written in python. A logic library will allow you to make inferences. Here's a simple inference:

```

>>> P = Variable("It is raining")
>>> Q = Variable("Ground is wet")
>>> R = IMPLIES(P, Q)
>>> entails(P, R)
FALSE
>>> entails(AND(P, R), Q)

```

TRUE

(You can think of “entailment” as “inference”.)

Q1B. This is an optional but important exercise. It is an intermediate step/practice between Q1 and Q2 where you write the actual CSP solver.

Modify `bt0` to get `bt1` in the following way. Note that when a variable V is assigned value v , you can create a new CSP where V is not a variable and all constraints containing V have all the V 's replaced by v .

For instance suppose a CSP has variables A, B, C , all with the same domain of $\{0, 1, 2\}$, and the constraints are:

$$\begin{aligned}A &< B \\ A + B &< C \\ B &< C + 2\end{aligned}$$

Suppose we (or rather a backtracking search) try assignment $A = 1$. Then the new CSP is made up of variables B, C (i.e., without A) and the constraints become

$$\begin{aligned}1 &< B \\ 1 + B &< C \\ B &< C + 2\end{aligned}$$

Note that in this case, if a constraint has no variables it should be removed. For instance if the continues with assignment $B = 2$, then the constraint $1 < B$ becomes $1 < 2$ and can be removed in the next CSP. Also, a solution is found (i.e., the assignment is complete) if there are no more variables in the resulting CSP.

Note that it's clear that if variables and constraints are removed from a CSP, the process to search for a solution will speed up since the new CSP has fewer variables to work with and fewer constraints to satisfy.

For instance in an n -queens problems where $n = 1000$, if 500 queens are already successfully placed and the variables corresponding these 500 queens and the constraints involving only these 500 queens are removed, the time to place the 501-st queen correctly will take less than if all the original constraints are retained.

Q2.

You need to now implement the CSP solver that implements the CSP backtrack search using FC, NC, AC with the MRV-MD and LCV heuristics. The relevant function is `bt2` in `bt2.py`. Most of the algorithmic steps to achieve this is really for CSP and not just sudoku. Therefore you will need to include methods in `CSP.py` to finish implementing `bt2`.

The following are some general comments relevant to the details inside the CSP class that you have to complete. (Also found in my notes on CSP.)

NODE CONSISTENCY

In your CSP class, add a method `node_consistent_check` such that after `csp.node_consistent_check(X)` (where `X` is a variable of `csp`) `csp` is changed in the following way:

The domain of `X` is replaced by the smaller list of values `x` such that $C(x)$ is true. (See CSP notes.)

Furthermore, if the domain of `X` becomes empty, then `False` is returned. Otherwise `True` is returned.

It is also convenient to write your method so that if `csp` calls `csp.node_consistent_check()` (i.e., no argument), then node consistency check is performed on *all* the variables of `csp`. In this case, if any domain becomes empty, `False` is returned. Otherwise `True` is returned.

Your code should look like this:

```
class CSP:
    ...
    def node_consistency_check(self, var=None):
        if var == None:
            vars = self.vars
        else:
            if var not in self.vars:
                raise ValueError("%s is not in self.vars" % var)
            vars = [var]
        for var in vars:
            ... PERFORM NODE CONSISTENCY CHECK FOR var ..
        ...
```

For instance if CSP has only one variable `X`, domain of `X` is

$$\{0, 1, 2, 3, 4\}$$

and the unary constraint on `X` is

$$C(X) = [X < 2]$$

then after node consistency check at `X`, the domain of `X` becomes

$$\{0, 1\}$$

Make sure you test this method thoroughly.

ARC CONSISTENCY CHECK

Write a method in your CSP class call `arc_consistency_check` such that when a CSP, `csp`, calls

```
csp.arc_consistency_check((X, Y))
```

where `X` and `Y` are variables in `csp`, `(X, Y)`-arc consistency check is performed, i.e., the domain of `X` is replaced by all `x`'s in the domain of `X` such that there is some `y` in the domain of `Y` satisfying `C(X=x, Y=y)` is `True`. Here `C` is the binary constraint containing `X` and `Y`.

The method returns `True` if the domain of `X` is changed (i.e., it shrunk) in the above process. Otherwise `False` is returned.

Test your method thoroughly!

NEIGHBORS AND DEGREE

(You'll need this in AC3 later).

If X and Y are variables that occur in the same constraint, then we say that Y is a neighbor of X (and vice versa). You will need this in AC3.

The degree of a variable X is the number of constraints that involves X (i.e., in the constraint hypergraph, it is the number of constraint nodes connected to X .) You should probably have a method to compute this. (You will need this for the MD heuristic.)

AC3

Add the method `ac3` to your `CSP` class.

```
class CSP:
    ...
    def ac3(self, arcs=None):
        if arcs is None:
            let arcs be a queue of all arcs, i.e., all (X,Y)
            where X and Y are variables of self

        while arc is not empty:
            remove arc (X, Y) from arcs
            perform arc consistency check on self with arc (X, Y)
            if the above check returns True (i.e., domain of X shrinks):
                if the domain of X is empty:
                    return False
                otherwise, add all (Z,X) to arcs if (Z,X) is not in arcs
                where Z is a neighbor of X
        return True
    ...
```

VARIABLE SELECTION

Add a method in your CSP class to select a variable with MRV minimum remaining values (i.e., smallest domain). However note that there might be more than one. In other words the method should return a list of such variables. Call the method `vars_with_mrv`. Test your method.

Add a method in your CSP class that access a set of variables and return those with the maximum degree.

Write a method `select_var` that accepts `option` and if `option['var_selection']` is MRV-MD, the method returns a variable using MRV-MD. Otherwise, the method returns the first variable in `var`.

Test your method thoroughly.

VALUE SELECTION

Instead of recursively performing recursive selection of value using LCV heuristic (see notes for LCV), we will order a domain by LCV and then perform a loop over the domain for value selection. (This will cut down on recursive function calls.) LCV means this: say variable is X . To compute how constraining is a value x in the domain of X , you look at the assignment $X = x$ and check against all neighbors of X and see how many values in their domain have to be removed. Add up all number of such values removed. The larger the number of values removed, the more constraining is the value x .

If you have more than two values that constraints in the minimal way, this is what you do: sort these values and pick the first. (In a real life application, you would randomize the choice.)

All in all, this means that for the given variable X , with domain D , you create a list $[\dots, (c, x), \dots]$ where x runs through D and c is the count of values that will be removed by the assignment $X = x$ on X 's neighbors. You then sort this list and extract the variables (i.e., ignore the counts c) to get the ordered domain.

Note that the method depends on a variable X (of course).

Altogether, `csp.ordered_dom(X)` returns the domain of X ordered by the LCV (and then by Python's ordering if there are ties).

Test thoroughly!!!

CSP BACKTRACK SEARCH

Now complete CSP solver using all the above.

Note that our CSP solver is a general CSP solver. Most of the work is done in the CSP class and not the SudokuCSP class. Therefore once the CSP class is completed, you can use it solve other CSP problems. As mentioned in class research in CSP is still pretty active. You will find many CSP papers if you google online. The code here can then be used for further investigations/studies/research.

NOTE. I leave it to you to decide if you want to write the various CSP optimization steps as methods in the CSP class or as functions. (If you write functions, then these should probably be kept in `CSP.py`.) For instance, instead of `csp.node_consistency_check(var)` you can also have `node_consistency_check(csp, var)`.

Q3. Write a CSP solver for the n -queens problem.